

Profiling-Guided Adaptive Optimization for Memory-Bound LLM Inference via Data Movement Analysis

Eugene Won

Information Coding and Processing Lab

Department of Electronic & Electrical Engineering, AI Software Engineering

Ewha Womans University

Seoul, Republic of Korea

eugenewon12@ewhain.net

Abstract—Large language model (LLM) inference is increasingly dominated by memory bandwidth limitations rather than computational capacity. In particular, autoregressive decoding exhibits low arithmetic intensity, making it memory-bound and leading to underutilization of modern GPUs. While optimization techniques such as operator fusion, mixed precision, and batching can improve inference performance, they are often applied statically without considering workload-dependent bottlenecks.

In this work, we present a profiling-guided framework that analyzes LLM inference from a data movement perspective and derives adaptive optimization strategies based on workload characteristics. By systematically measuring latency, throughput, GPU utilization, memory usage, and arithmetic intensity across varying batch sizes and sequence lengths, we characterize bottleneck regimes and identify how different optimizations affect effective memory traffic. Based on these observations, we propose a workload-aware adaptive policy that selects optimization strategies according to runtime conditions. Our approach aims to provide practical insights into memory-centric AI system design and workload-aware LLM serving optimization.

I. INTRODUCTION

Recent advances in large language models (LLMs) have dramatically increased computational requirements, but real-world inference performance is often limited by memory bandwidth rather than peak compute capability. In particular, the autoregressive decoding phase repeatedly accesses model weights and key-value cache states from GPU memory, resulting in low arithmetic intensity and high data movement.

Under practical deployment settings, such as small batch sizes and latency-sensitive inference, GPU utilization often remains low despite high theoretical FLOPS. This inefficiency arises from kernel launch overhead, insufficient parallelism, repeated memory access, and workload-dependent execution behavior.

In particular, batch-1 inference often suffers from severe kernel launch overhead, which can become a dominant bottleneck rather than memory bandwidth. We hypothesize that operator fusion through `torch.compile` can reduce this overhead for many low-utilization workloads by collapsing multiple small kernel launches and reducing redundant runtime overhead. However, the benefit of `torch.compile` is

workload- and architecture-dependent, and it does not universally outperform mixed precision in every low-utilization case.

Although optimization techniques such as mixed precision, operator fusion, memory-efficient attention, continuous batching, and specialized inference runtimes have been widely studied, they are often applied as static configurations. However, the dominant bottleneck shifts depending on batch size, sequence length, and runtime workload characteristics.

In this work, we adopt a system-level perspective and analyze LLM inference through the lens of memory-centric AI. Rather than focusing only on computation reduction, we investigate how optimization strategies affect data movement, GPU utilization, arithmetic intensity, and workload-dependent performance behavior.

II. RELATED WORK

LLM inference optimization has been studied from several perspectives, including attention kernel optimization, memory management, batching, scheduling, and runtime-level acceleration.

FlashAttention and FlashAttention-2 improve attention computation by reducing high-bandwidth memory access and improving parallelism through IO-aware tiling and better work partitioning. vLLM introduces PagedAttention, which improves key-value cache memory management for LLM serving. TensorRT-LLM provides GPU-optimized inference kernels and runtime-level acceleration for transformer models.

Speculative decoding accelerates autoregressive generation by using a smaller draft model to propose multiple tokens and verifying them with a larger target model. Orca improves serving efficiency for transformer-based generative models through iteration-level scheduling. Continuous batching further improves serving throughput by dynamically batching requests at token-level granularity rather than using static request-level batching. SARATHI [11] improves serving efficiency by piggybacking decodes with chunked prefills, reducing pipeline bubbles in multi-stage inference.

DeepSpeed-Inference optimizes transformer inference through kernel fusion and memory-aware execution. FlexGen

explores high-throughput generative inference under limited GPU memory by coordinating GPU, CPU, and disk memory.

The roofline model [6] provides a principled framework for characterizing memory-bound versus compute-bound behavior based on arithmetic intensity and hardware ceilings. We adopt this perspective to analyze how workload characteristics influence inference bottlenecks.

However, existing approaches generally apply optimizations statically without explicitly considering workload-dependent bottleneck transitions. In contrast, our work proposes a profiling-guided adaptive framework that selects optimization strategies according to measured runtime regimes.

III. PROPOSED METHOD

We propose a profiling-guided adaptive optimization framework that analyzes LLM inference bottlenecks and selects optimization strategies based on runtime workload characteristics.

A. Profiling and Bottleneck Characterization

We collect runtime metrics using PyTorch Profiler and GPU monitoring tools, including latency, throughput, kernel execution time, GPU utilization, memory usage, estimated memory bandwidth, and arithmetic intensity.

We classify execution regimes using an exhaustive rule based on measured GPU utilization and sequence length:

- **Low-utilization regime:** GPU utilization $< 35\%$. SM activity is insufficient, and kernel launch latency and insufficient parallelism dominate.
- **Kernel-overhead-bound regime:** All workloads that do not satisfy the low-utilization or memory-bound conditions. This includes cases where GPU utilization $\geq 90\%$ with sequence length < 256 . In this regime, attention FLOPs are not yet dominant, and kernel launch overhead still contributes significantly to per-token latency.
- **Memory-bound regime:** GPU utilization $\geq 90\%$ with sequence length ≥ 256 . In this regime, long-sequence attention and repeated memory access increase HBM pressure, making memory traffic a dominant bottleneck.

Note that on TITAN Xp, pynvml-reported GPU utilization measures SM active time, which can rise at large batch sizes and long sequence lengths because attention FLOPs, scaling as $O(\text{seq}^2 \times \text{batch})$, increasingly occupy the SMs even as HBM stalls occur. Empirically, this combination of high SM activity and long-sequence memory access correlates with HBM saturation, which we validate using estimated memory bandwidth approaching the TITAN Xp hardware ceiling of 547.7 GB/s.

GPT-2 Large is not observed in the low-utilization regime under our threshold because its larger parameter count results in GPU utilization above 35% even at batch size 1. Therefore, GPT-2 Large workloads are classified as kernel-overhead-bound rather than low-utilization-bound in our regime analysis.

To better understand memory-bound behavior, we additionally analyze workloads using arithmetic intensity and roofline-based interpretation. For the decode phase, arithmetic intensity is computed as

$$\text{AI} = \frac{2 \times N \times B}{W_{\text{bytes}}}$$

FLOPs/byte, where N is the parameter count, B is the batch size, and W_{bytes} is the model weight size in bytes.

These thresholds will be validated empirically by sweeping batch sizes from 1 to 32 and sequence lengths from 32 to 512 while correlating GPU utilization, estimated memory bandwidth pressure, arithmetic intensity, and kernel launch behavior.

B. Optimization Strategies

We evaluate practical system-level optimizations that influence memory traffic, kernel overhead, and execution efficiency:

- Operator fusion via `torch.compile` to reduce kernel launch overhead and redundant memory access.
- Mixed precision execution using FP16 to reduce memory footprint and bandwidth usage.
- Batch size scaling to increase arithmetic intensity and theoretical parallelism.
- Profiling-guided configuration selection based on observed bottleneck regimes.

Note that batch-size scaling, while theoretically beneficial, is not enforced by the adaptive policy at runtime because it would alter request scheduling and potentially violate latency constraints in serving environments.

C. Adaptive Optimization Policy

Based on our profiling methodology, we derive the following adaptive policy:

- **Low-utilization regime:** Apply operator fusion via `torch.compile`.
At small batch sizes, GPU SM activity is expected to remain low, and kernel launch latency is likely to dominate per-token inference cost. Operator fusion can collapse multiple small kernel launches and reduce redundant runtime overhead. In our analysis, `torch.compile` provides the largest average improvement in this regime, although FP16 can outperform `torch.compile` in some GPT-2 workloads.

- **Kernel-overhead-bound regime:** Apply `torch.compile`.

In intermediate or short-sequence high-utilization regimes, kernel launch overhead is still expected to contribute significantly relative to long-sequence memory traffic. Operator fusion is therefore expected to improve execution efficiency.

- **Memory-bound regime:** Apply FP16 mixed precision. When long sequence lengths and high SM activity correlate with HBM bandwidth saturation, reducing model weight precision via FP16 is expected to reduce memory traffic and bandwidth pressure.

We expect this profiling-guided policy to outperform any single static optimization strategy in many workload conditions because each optimization decision is conditioned on the measured runtime bottleneck rather than applied uniformly. However, the policy is not intended to be universally optimal across all model architectures and regimes.

TABLE I
PROFILING-GUIDED ADAPTIVE OPTIMIZATION POLICY.

Regime	Condition	Optimization	Expected Benefit
Low-utilization	GPU util. < 35%	<code>torch.compile</code>	Reduce kernel launch overhead
Kernel-overhead-bound	Otherwise	<code>torch.compile</code>	Fuse redundant kernel launches
Memory-bound	GPU util. $\geq 90\%$ and seq. ≥ 256	FP16	Reduce HBM traffic

IV. EXPERIMENTAL PLAN

Experiments will be conducted on a multi-GPU server equipped with four NVIDIA TITAN Xp GPUs. Each GPU has 12GB of memory. The software environment uses NVIDIA driver version 535.230.02 and CUDA version 12.2.

All experiments are conducted on a single GPU; the remaining GPUs are reserved for future multi-GPU scaling experiments.

The TITAN Xp GPU provides approximately 12.15 TFLOP/s peak FP32 performance and approximately 547.7 GB/s peak memory bandwidth. These hardware-specific values will be used for roofline-based interpretation of compute-bound and memory-bound behavior.

We plan to evaluate GPT-2 Small (124M), GPT-2 Large (762M), and GPT-Neo (125M) as primary models under varying batch sizes and sequence lengths. We additionally evaluate GPT-2 Small, OPT-125M, OPT-350M, and BLOOM-560M for generalization analysis across multiple autoregressive transformer architectures.

The goal is to analyze how workload characteristics influence latency, throughput, GPU utilization, arithmetic intensity, and memory behavior.

We compare:

- Baseline inference
- Mixed precision inference using FP16
- Operator fusion using `torch.compile`
- Profiling-guided adaptive optimization

We measure:

- Latency (ms/token)
- Throughput (tokens/sec)
- GPU utilization
- Memory usage
- Estimated memory bandwidth
- Arithmetic intensity
- Kernel execution behavior

Additionally, we analyze how performance changes as workloads transition between low-utilization, kernel-overhead-bound, memory-bound, and compute-bound regimes by varying batch size and sequence length.

A. Ablation Study

To validate the necessity of regime-based policy selection, we plan to conduct the following ablation studies:

- **Wrong-optimization ablation:** Apply each optimization in mismatched regimes and measure throughput degradation.
- **Rule-removal ablation:** Compare static policies, including `always-torch.compile` and `always-FP16`, against the adaptive policy across varying workload conditions.
- **Prompt-diversity analysis:** Repeat experiments using prompts from multiple categories including code generation, question answering, story generation, chat, and mathematical reasoning to evaluate whether regime boundaries remain consistent across workload types.

B. Generalization

To evaluate architecture independence, we additionally plan to test the proposed framework across multiple model families including GPT-2 Small, OPT-125M, OPT-350M, GPT-Neo-125M, and BLOOM-560M.

We expect regime transitions and optimization trends to remain broadly consistent across architectures that share similar autoregressive decoding behavior and memory access patterns. However, the effectiveness of each optimization is not identical across architectures. In particular, some models show limited generalization of the adaptive policy: BLOOM-560M and GPT-Neo-125M can underperform the baseline on average or in specific workload regimes. For GPT-Neo-125M, the low-utilization regime shows especially limited benefit from `torch.compile` in some cases, indicating that the proposed policy should be interpreted as a profiling-guided heuristic rather than a universally optimal rule.

These results suggest that architecture-specific kernel behavior, implementation details, and model structure can affect whether operator fusion or mixed precision provides the strongest improvement. Therefore, the adaptive policy is most useful when calibrated through lightweight profiling on the target model and hardware.

V. EXPECTED CONTRIBUTIONS

This work aims to provide:

- A memory-centric analysis framework for understanding LLM inference bottlenecks.
- A profiling-guided methodology for classifying workload-dependent execution regimes.
- A practical workload-aware adaptive optimization policy for LLM serving.
- Insights into how arithmetic intensity, data movement, and kernel overhead influence inference performance.
- A roofline-based interpretation of memory-bound and compute-bound behavior in autoregressive decoding.
- An empirical discussion of the generalization limits of rule-based adaptive optimization across different transformer architectures.

Our approach emphasizes that improving AI system performance requires understanding runtime bottlenecks and reducing unnecessary data movement rather than focusing solely on computational throughput.

VI. CONCLUSION

We propose a profiling-guided adaptive optimization framework for LLM inference. By analyzing execution through GPU utilization, arithmetic intensity, and data movement, our approach aims to identify workload-dependent bottleneck regimes and apply appropriate optimization strategies.

The proposed policy shows that workload-aware optimization can be more informative than applying a single static configuration. At the same time, our generalization analysis highlights that architecture-specific behavior can limit the effectiveness of a simple rule-based policy. Therefore, profiling-guided calibration remains necessary when applying the framework to new models or hardware platforms.

We expect this framework to provide practical insights into memory-centric AI system design and workload-aware inference optimization, particularly for latency-sensitive LLM serving environments.

REFERENCES

- [1] PyTorch Team, “TorchDynamo and TorchInductor,” 2023. [Online]. Available: <https://pytorch.org/get-started/pytorch-2.0/>
- [2] T. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” *NeurIPS*, 2022.
- [3] T. Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning,” *ICLR*, 2024.
- [4] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” *SOSP*, 2023.
- [5] G. Yu et al., “Orca: A Distributed Serving System for Transformer-Based Generative Models,” *OSDI*, 2022.
- [6] S. Williams et al., “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *CACM*, 2009.
- [7] R. Y. Aminabadi et al., “DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale,” *SC*, 2022.
- [8] Y. Leviathan et al., “Fast Inference from Transformers via Speculative Decoding,” *ICML*, 2023.
- [9] Y. Sheng et al., “FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU,” *ICML*, 2023.
- [10] NVIDIA, “TensorRT-LLM: Optimizing Large Language Model Inference on GPUs,” 2023. [Online]. Available: <https://developer.nvidia.com/tensorrt>
- [11] S. Agrawal et al., “SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills,” 2023.
- [12] Anyscale, “Continuous Batching: How vLLM Increases LLM Throughput,” 2023. [Online]. Available: <https://www.anyscale.com/blog/continuous-batching-llm-inference>